Written 4

- Due Apr 30 at 10pm
- Points 110
- Questions 17
- Available until May 1 at 10pm
- Time Limit None

Instructions

Read chapter 4 of the textbook and complete the following exercises:

- 4.10
- 4.11
- 4.15
- 4.22
- 4.25

You may type your work into the boxes in the quiz or upload individual images or PDFs for each answer. You may not upload a single PDF or PDFs that cover multiple answers. The TAs have complained about the collective disorganization of submissions in past semesters and begged the instructores for this respite (from single PDF uploads).

You must show all work. Answers without work will receive no credit. Answers with insufficient work will receive little to no credit

Please keep your work neat and organized. The TAs will penalize your grade if they are forced to expend undue effort to decipher cryptic submissions

This quiz was locked May 1 at 10pm.

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	106 minutes	104 out of 110

Score for this quiz: 104 out of 110
Submitted Apr 30 at 9:56pm
This attempt took 106 minutes.

Question 1
5 / 5 pts

4.10.1

Your Answer:

4.1) Doubling Registers 32 → 64

4.10.1) original cycle time ⇒ 30 + 250 + 150 + 25 + 200 + 250 + 25 + 20
⇒ 950 ps

New cycle time → [Read latency ⇒ 150 to 160]
→ 960 ps

memory instructions ⇒ 35 %. ; 12 %. reduction :

$$IC_{new} = IC_{orig} \times (1 - (0.35 \times 0.12)) = 0.958 \times IC_{orig}$$

$$\text{Speedup} = \frac{IC_{orig} \times T_{orig}}{IC_{new} \times T_{new}} = \frac{1 \times 950}{0.958 \times 960} = 1.032$$

Ans) 3.2 % faster

Question 2
7 / 10 pts

4.10.2

Your Answer:

4.10.2) Base cost :- 1000 + 2000 + 200 + 100 + (2 × 30) + (4 × 10) + 100 + 500 + 5 + 2
= 4007

After doubling register file, cost ⇒ 4007 - 200 + 400 ⇒ 4207

cost Increase :- $\frac{200}{4007}$ = 5.0 %.

comparison ⇒ performance = + 3.2 %.
cost = + 5.0 %.

Performance gained per Dollar = 3.2 %. / 5.0 %. = 0.64

∴ cost increases more than performance
performance to cost ratio decreases

Base cost calculation is missing a control unit. (-3)

Question 3
10 / 10 pts

4.10.3

Your Answer:

4.10.3) Add if → Performance is the main priority and cost and size important
Examples :- Servers, Scientific computers.
Don't add if → We are designing something which is cost or power constrained.
Examples :- Phones

Question 4
5 / 5 pts

4.11.1

Your Answer:

4.11) Reg[Rd] = Mem[Reg[Rm] + Reg[Rn]]

4.11.1) No new functional blocks are needed because :
→ The reg file already has 2 read, 1 write port
→ ALU can add Reg[Rn] + Reg[Rm]
→ Data memory can perform the load

Question 5
5 / 5 pts

4.11.2

Your Answer:

4.11.2) No block requires modifications because the existing datapath can handle LWI without any structural changes.

Only the control unit needs modification.It must recognize the new LWI opcode and set the correct control signals. So modification may or may not be needed depending on how opcode isstructured

Question 6
5 / 5 pts

4.11.3

Your Answer:

4.11.3) No new data paths are needed.
The R-Type path [ReadData2 → ALU → Memory address] already exists.

Question 7
5 / 5 pts

4.11.4

Your Answer:

4.11.4) No new control signals are required. We use a new combination of the existing signals:
Reg2Loc = 0 ; ALUSrc = 0 [use register operands]
ALUOp = 00 (add), MemRead = 1, MemWrite = 0
MemtoReg = 1 ; RegWrite = 1

Question 8
5 / 5 pts

4.15.1

Your Answer:

4.15.1) LDUR doesn't use ALU.
$T_{new} = 30 + 250 + 150 + 250 + 25 + 20 = 725$ ps
(Ans)

Question 9
10 / 10 pts

4.15.2

Your Answer:

4.15.2) New IC = 1 + 0.35 = 1.35

Execution Time =>

Old = 950N ; New = 1.35N × 725 = 978.75N

Speedup => $\frac{950}{978.75} = 0.9706$

Ans) 3 % slower

Original CPU time = 925n New CPU time = 725n New instruction count = 1.35n New time = 1.35n * 725 = 978.75n Speedup = Original CPU time / New CPU time ≈ 0.945 (slower) Logical steps and intermediate calculations are clear. The answer would be correct if previous calculations are correct.

Question 10
5 / 5 pts

4.15.3

Your Answer:

4.15.3) The key factor is percentage of memory instructions [LOUR / STUR] If the program has a lot of these instructions, the increase in instruction count worsens the performance more than the faster clock cycle improves it.

Question 11
5 / 5 pts

4.15.4

Your Answer:

4.15.4) The original CPU design is better because:
→ lower instruction count
→ Better overall performance
→ Simpler instruction usage
The 35 % increase in instruction count cancells the 23.7 % reduction in clock period resulting in an overall slower execution in the new CPU.

Question 12
5 / 5 pts

4.22.1

Your Answer:

4.22.1) STUR and LDUR use MEM stage for data memory, during those cycles, IF stalls.

cycle 4 : I1 in MEM → IF stalls
cycle 5 : I2 in MEM → IF stalls again
→ ∴ 2 stall cycles total

Question 13
5 / 5 pts

4.22.2

Your Answer:

4.22.2) Yes, placing non-memory instructions between memory instructions can reduce stalls but it wont eliminate all stalls in general.
By pushing the conflicted memory access to a cycle when no new instructions needs to be fetched, we can reduce the number of stalls.

Question 14
4 / 5 pts

4.22.3

Your Answer:

4.22.3) Yes, Inserting a NOP delays instruction fetch. This shifts IF so it does not overlap with MEM accel.
∴ It can be handled in software using NOPs, but its ineffecient.

You can’t solve this structural hazard with NOPs, because even the NOPs must be fetched from instruction memory.

Question 15
5 / 5 pts

4.22.4

Your Answer:

4.22.4) If 35 % of the instructions are memory-references, and each causes 1 stall,
The CPI increases from 1 to 1.35.
The processor spends 35 % of its time stalling.

Question 16
10 / 10 pts

4.25.1

Your Answer:

4.25.1) load-use hazard between LDUR X11 -> ADD X12
Pipeline Diagram :-
[ST => Stall]

cycle	1	2	3	4	5	6	7	8	9	10	11	12	13
A 1	IF	ID	EX	MEM	WB								
B 1		IF	ID	EX	MEM	WB							
C 1			IF	ID	ST	EX	MEM	WB					
D 1				IF	ST	ID	EX	MEM	WB				
E 1					--	IF	ID	EX	MEM				
A 2						IF	ID	EX	MEM				
B 2							IF	ID	EX	WB			
C 2								IF	ID	EX	MEM	WB	
D 2								IF	ST	EX	EX	MEM	WB
E 2									--	ID	EX	MEM	WB

Question 17
8 / 10 pts

4.25.2

Your Answer:

4.25.2) Each loop iteration has :-
5 instructions
+1 stall cycle

Total cycles per iter = 6

The stall introduces a single bubble
The bubble propagates through the pipeline stages

Only 1 out of every 6 cycles has a stall

∴ most cycles have all stages doing useful work

there will be no fully utilized cycle b/c some of previous instructions aren't utilizing every stage like SUBI doesn't use ME and CBNZ doesn't use ME & WB, this will cause later cycle 8,9,10 to not be fully utilized.

Quiz Score: 104 out of 110